HASHMAPS IN JAVA

Day-wise Revision Notes • Fundamentals → Problem Solving

Learning goals

• Understand key–value storage and when a HashMap is useful.

• Use common HashMap methods safely in Java.

• Solve frequency-counting and Two Sum problems.

• Explain hashing, collisions, and time complexity in interviews.

1. What is a HashMap?

A **HashMap** stores data as **key–value pairs**. Think of a contact list: a person's name is the key and the phone number is the value. HashMap belongs to Java Collections Framework and is available through `java.util`.

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> marks = new HashMap<>();
marks.put("Arun", 85);
marks.put("Meena", 92);
System.out.println(marks.get("Arun")); // 85
```

Important properties

• Keys are unique; inserting an existing key replaces its old value.

• Values may repeat. HashMap allows one null key and multiple null values.

• Iteration order is not guaranteed—do not rely on insertion or sorted order.

• put/get/remove are expected O(1) on average; worst-case can be slower.

• HashMap is not synchronized for concurrent mutation.

When should you think of a HashMap?

Use it for fast lookup by identifier, occurrence counting, remembering previously seen values, or grouping information by a key.

2. Core methods

Method	Meaning	Example
put(k, v)	Insert or replace value	map.put("A", 10)
get(k)	Get value; null if absent (or mapped to null)	map.get("A")
getOrDefault(k, d)	Return value or default	map.getOrDefault("B", 0)
containsKey(k)	Check whether key exists	map.containsKey("A")
remove(k)	Delete mapping	map.remove("A")
size()	Number of mappings	map.size()
keySet()	View of keys	map.keySet()
entrySet()	View of key-value entries	map.entrySet()

Frequency-counting pattern

Increase the count each time an element appears.

```
int[] nums = {2, 3, 2, 5, 3, 2};
Map<Integer, Integer> freq = new HashMap<>();
for (int n : nums) {
    freq.put(n, freq.getOrDefault(n, 0) + 1);
}
System.out.println(freq); // counts: 2→3, 3→2, 5→1
```

Why getOrDefault? If a key has not appeared, treat its count as zero and add one. It avoids a separate containsKey check.

Safe lookup note

get() can return null. Avoid unboxing a missing Integer without checking. For frequency counting, getOrDefault() is usually the cleanest choice.

3. Problem 1 — Count frequency of each number

Problem: Given an integer array, print each distinct number and its frequency. Output order does not matter.

Sample input

```
6
4 2 4 7 2 4
```

Sample output (one valid order)

```
4 -> 3
2 -> 2
7 -> 1
```

Approach

• Create a map: number → count.

• Traverse the array and increase the count (or start from zero).

• Traverse entrySet() to print each key and count.

Java solution

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<Integer, Integer> freq = new HashMap<>();

        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }
        sc.close();
    }
}
```

Complexity: Expected O(n) time and O(k) extra space, where n is array length and k is the number of distinct values.

Trace

Read	Map after processing
4	{4=1}
2	{4=1, 2=1}
4	{4=2, 2=1}
7	{4=2, 2=1, 7=1}
2	{4=2, 2=2, 7=1}
4	{4=3, 2=2, 7=1}

4. Problem 2 — Two Sum

Problem: Given an integer array and target, return indices of two different elements whose sum equals target. Assume exactly one answer exists.

Sample input

```
4
2 7 11 15
9
```

Sample output

```
0 1
```

HashMap idea

For each number x, the required partner is target - x. Check whether that partner has already been seen. The map stores number → index.

Java solution

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int target = sc.nextInt();

        Map<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int need = target - a[i];
            if (seen.containsKey(need)) {
                System.out.println(seen.get(need) + " " + i);
                sc.close();
                return;
            }
            seen.put(a[i], i);
        }
        System.out.println("No pair found");
        sc.close();
    }
}
```

Dry run

• Index 0: x = 2, need = 7. No 7 yet; store 2 → 0.

• Index 1: x = 7, need = 2. The map contains 2, so output indices 0 and 1.

> **Complexity:** Expected O(n) time and O(n) extra space. Nested loops take O(n²); a HashMap enables one-pass lookup.

5. Hashing and collisions

HashMap uses a key's hashCode() to help choose a bucket, then equals() to identify the matching key. A **collision** occurs when different keys map to the same bucket. Java handles collisions internally. Equal keys must have equal hash codes. For custom key classes, implement equals() and hashCode() consistently.

6. Common mistakes & interview checklist

• **Assuming order:** HashMap does not promise sorted or insertion order. Use TreeMap for sorted keys or LinkedHashMap for predictable iteration.

• **Duplicate keys:** put(key, newValue) replaces the old value; it does not add a second entry.

• **containsValue vs containsKey:** key lookup is usually the common and more efficient operation.

• **Mutable keys:** avoid changing fields used by equals/hashCode while an object is a key.

• **Two Sum order:** check the complement before inserting the current value to avoid using the same element twice.

• **O(1) nuance:** it is expected/average performance, not a guarantee for every input.

Quick comparison

Collection	Best fit	Ordering
HashMap	Fast key-value lookup	No guaranteed order
LinkedHashMap	Lookup + predictable iteration	Predictable iteration order
TreeMap	Sorted keys	Sorted by key

Practice questions

• Count the frequency of each character in a string.

• Find the first non-repeating character.

• Check whether two strings are anagrams using character counts.

• Find the single element when others appear twice; compare counting with XOR.

• Group words that are anagrams of each other.

Interview Aspect — 30-second answer

> “HashMap stores unique keys mapped to values. It uses hashing to locate buckets and equals() to check key identity. put, get, and remove are expected O(1) on average, though collisions can affect performance. It permits one null key, does not guarantee iteration order, and is not synchronized.”

Revision checklist

■ Explain key vs value ■ Use getOrDefault() ■ Build a frequency map ■ Solve Two Sum ■ Explain collisions and equals/hashCode ■ State time and space complexity.